AI AGENTS, MCP & APIS FOR PMS AND BAS · LESSON 10 OF 11

Agent Frameworks: LangGraph, CrewAI and AutoGen

Compare the three agent frameworks most often named in job descriptions. Learn LangGraph's stateful graphs with checkpoints and human approval, CrewAI's role-based crews for quick prototypes, and AutoGen's conversational multi-agent concept, and choose a framework by the control and reliability a task needs.

Gap-fill lesson | 2026 | Advanced | 120 min

By the end of this lesson you can

- Explain nodes, edges and state in a LangGraph workflow
- Describe a CrewAI crew of role-based agents and when to use it
- Explain the conversational multi-agent model used by AutoGen
- Choose a framework by control, reliability and maintenance needs

Why frameworks exist

An agent needs a loop (call the model, run a tool, check the result), memory of what has happened, a way to stop, and often retries and human approval. Writing that from scratch is possible (a small loop is shown in the Grokbot course), but production systems need the same features every time. Frameworks provide them. The three you will meet most often take different approaches.

CASE STUDY **A refund workflow that must not lose its place**

A refund process has four steps: check the order, check the policy, ask a manager to approve, then issue the refund. If the server restarts after approval but before the refund, the process must resume exactly where it stopped, without refunding twice. That requirement decides the framework.

LangGraph: stateful agents as graphs

LangGraph (part of the LangChain ecosystem) models an agent as a **graph**:

- **Nodes** are steps, such as "check order" or "call the model".
- **Edges** decide what runs next, including conditional edges ("if approved, go to refund; otherwise, end").
- **State** is a shared record that every node reads and updates, and it is **saved between steps** (checkpoints).

Python

```python
# Illustrative LangGraph-style structure (not run; check the current LangGraph
documentation,
# because the API changes between versions)
from typing import TypedDict

class RefundState(TypedDict):
    order_id: str
    eligible: bool
    approved: bool
    refunded: bool

def check_policy(state: RefundState) -> RefundState:
    state["eligible"] = state["order_id"].startswith("ORD")   # placeholder rule
    return state

def route_after_policy(state: RefundState) -> str:
    return "ask_approval" if state["eligible"] else "end"

# graph.add_node("check_policy", check_policy)
# graph.add_conditional_edges("check_policy", route_after_policy, {...})
# graph.compile(checkpointer=...)   # saved state lets the run resume after a restart
```

Use LangGraph when you need **control**: explicit steps, retries at each node, checkpoints so a long job resumes, and a clear place to insert human approval. The cost is more code and a steeper learning curve.

::::note Interrupt for approval LangGraph-style workflows can pause before a sensitive node, wait for a person's decision stored in the state, then continue. This is the cleanest way to implement the approval step from the MCP lesson. Check the current documentation for the exact interrupt API. ::::

CrewAI: role-based agent teams

CrewAI describes a team of agents, each with a **role**, a **goal** and a set of **tasks**. A **crew** runs the tasks in sequence, or under a manager agent that assigns them.

Python

```
# Illustrative CrewAI-style definition (not run; confirm names in the current docs)
# researcher = Agent(role="Market researcher", goal="Summarise competitor pricing", tools=
[...])
# writer     = Agent(role="Copywriter", goal="Draft a one-page brief from the research")
# research   = Task(description="List competitor prices", agent=researcher)
# brief      = Task(description="Write the brief", agent=writer, context=[research])
# crew = Crew(agents=[researcher, writer], tasks=[research, brief], process="sequential")
# crew.kickoff()
```

CrewAI is quick to prototype because the roles read like a job description. It suits content and research pipelines. It gives less control over exact state and checkpoints than LangGraph, so check how a crew behaves when a step fails before you rely on it in production.

AutoGen: conversational multi-agent systems

AutoGen (from Microsoft Research) models agents as participants in a **conversation**: they send messages to each other until the task is done, optionally with a human in the conversation. You should understand the concept even if you choose a different framework, because many multi-agent designs are variations of it: a planner messages a coder, the coder messages a tester, and the loop continues until the tests pass or a limit is reached.

Check the project's official documentation for the current version and its API, which has changed as the project has developed.

Comparing the three

Question	LangGraph	CrewAI	AutoGen
Core model	Graph of steps with shared state	Crew of role-based agents and tasks	Agents exchanging messages
Best for	Reliable, controlled, long-running workflows	Quick prototypes and content pipelines	Multi-agent conversations, code and tests
Saved state and resume	Yes, through checkpoints	Limited; check the current docs	Conversation history
Human approval	Clear interrupt points	Possible through task design	Possible as a participant
Learning curve	Higher	Lower	Medium

How to choose

1. Write the workflow as steps on paper. If the steps are fixed and need resume and approval, use a graph (LangGraph) or a plain workflow tool (n8n) instead of an agent.
2. If you need a team of roles to draft and review content quickly, try CrewAI and measure the result.
3. If the task is a conversation between specialists that must check each other's work, study AutoGen's pattern.
4. Whatever you choose, add tracing, a step limit, a budget and the approval steps from the MCP and OAuth lessons. A framework does not make an agent safe.

::::warn Frameworks change quickly Agent frameworks release breaking changes often. Pin versions in your project, read the release notes before upgrading, and treat any code sample older than a few months as a starting point to check, not a finished recipe. ::::

INTERVIEW CORNER

When would you choose LangGraph over a simple loop?

When the process has several steps with branches, must survive restarts, needs an approval pause, or must retry individual steps without repeating the whole job. A simple loop is enough when the task is a single tool-using conversation with a step limit.

::::key What to remember - LangGraph: nodes, edges and checkpointed state; best for control and reliability. - CrewAI: role-based crews; quick to prototype, less control over state. - AutoGen: agents exchanging messages; the concept matters even if you use another tool. - Choose by the control, resume and approval the task needs, and pin framework versions. ::::

Quick quiz

5 questions · tap an answer to check it

1. **In LangGraph, what is saved between steps so that a run can resume?**

A The model weights

B The checkpointed state

C The user's password

D The chart colours

Answer: B. Checkpoints store the shared state, so a restarted run continues from the last step.

2. **Which framework is described as role-based teams with goals and tasks?**

A LangGraph

B CrewAI

C AutoGen

D Pandas

Answer: B. A crew is made of agents with roles, goals and tasks.

3. **A refund process must not refund twice after a restart. Which property is most important?**

A Speed

B Idempotent steps with saved state

C Colourful output

D Many agents

Answer: B. Resuming safely needs saved state and actions that are safe to repeat.

4. **What is the key idea of AutoGen-style systems?**

A A single long prompt

B Agents exchanging messages until the task is complete

C Storing data in a graph

D Replacing the model

Answer: B. Conversation between specialised agents is the core pattern.

5. Why pin framework versions?

A They are free only when pinned

B Breaking changes between releases can silently change behaviour

C Pinning makes models smarter

D It disables tools

Answer: B. Versions change APIs; pinning keeps the system stable.

EXERCISE **Pick a framework**

For one workflow you know (for example, a weekly sales report with an approval step), write the steps, mark each one as deterministic, model-driven or needing a person, and choose LangGraph, CrewAI, AutoGen or a plain workflow tool. Justify the choice in five sentences, and list the three controls you would add whatever you chose.

Show solution (worked answer)

Hints: a workflow with no model decisions does not need an agent framework at all. Where a model chooses between branches, a graph with conditional edges is usually the clearest design. Name the approval node and the state it reads.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.